# Databases and Advanced Data Techniques Midterm: Earth-like Exoplanets

## Stage 1: Find and critique a dataset

### Intro

When it came to choosing what kind of data was wanted, it was already known that it would be related to astronomy. Astronomy data was chosen before for previous projects visualizing and interpreting galaxy data. This time, planet research seemed like the perfect fit for a relational database because of how we've organized the universe. Planets are part of solar systems, and they are part of galaxies, and they are part of clusters, and they are part of the early universe or the late universe. With some searching, the public NASA exoplanet archive operated by Caltech, under contract with NASA under the Exoplanet Exploration Program, was found (high discoverability), and it felt perfect for the focus of this project, to look for Earth-like planets, not just habitable planets, but ones similar to Earth in as many data points as possible. After getting to the main page for the exoplanet archive, planetary systems and composite data were at the bottom, and in there was a large table with hundreds of columns and rows about planets, easily discoverable and easy to acquire for personal projects through the "Download Table" tab, top left. The data was seen as free and open to the public. Since the focus was on Earth-like planets, constraints were used one by one to see how many planets we could work with. After setting three constraints, radius, equilibrium temperature, and stellar effective temperature, what started as thousands of planets turned into three planets, and all with missing data. If the constraints were too similar to Earth, we would be left with zero data. It was realized that if a database project about Earth-like planets was going to be done, a different approach was needed. So instead, we left out the constraints and downloaded the data as is. We started with two datasets. One for the abbreviations of the columns and the other for the exoplanet data. We started with 320 columns and 6158 planets. The rest, like the cleaning and preprocessing, would be left with the Jupyter notebook. If we start with as much data as possible and then clean up the data in Jupyter, then we'll have more to work with. Additionally, we can choose to balance between complete data and earth-like parameters or find a happy medium where we have complete data and earth-like planets with multiple constraints.

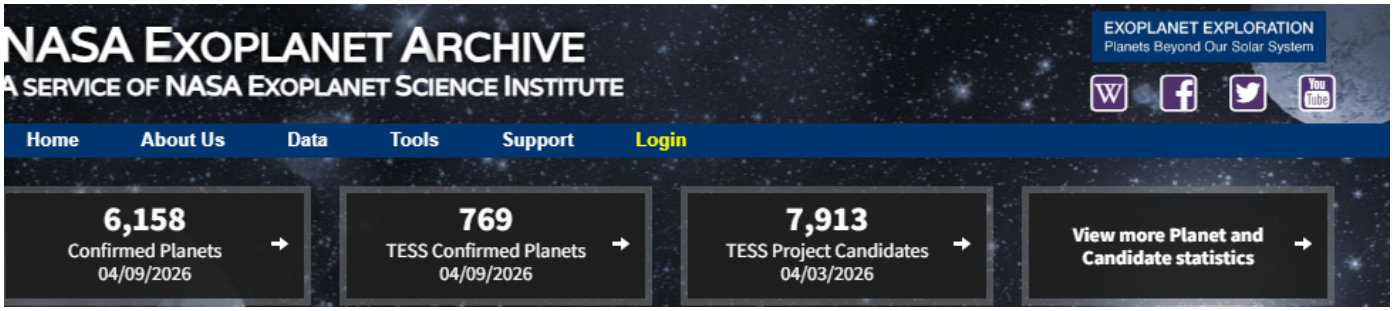
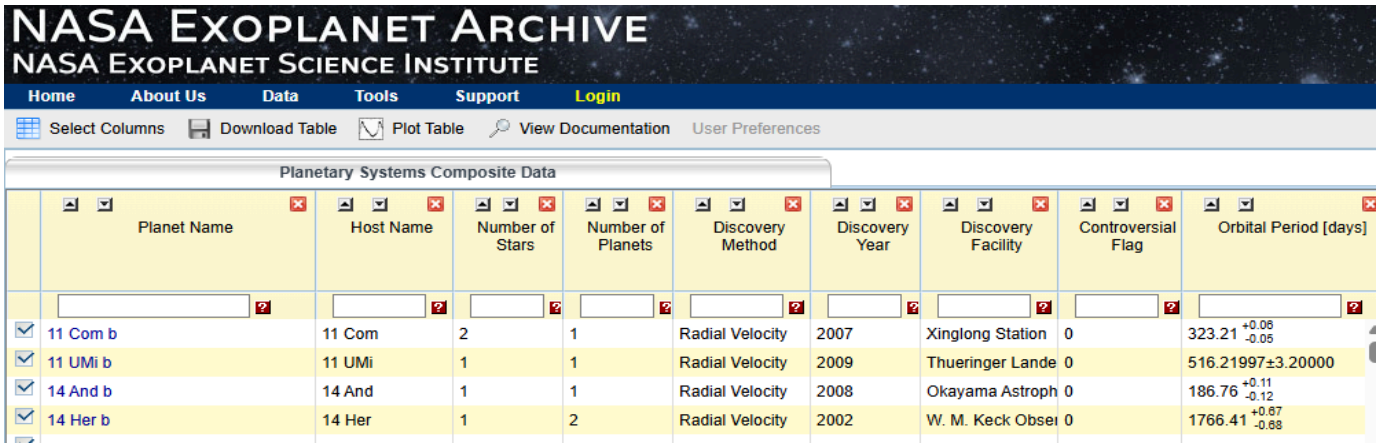



Figure 1. Top of the main page for the exoplanet archive




| Planet Name | Host Name | Number of Stars | Number of Planets | Discovery Method | Discovery Year | Discovery Facility | Controversial Flag | Orbital Period [days] |
|---|---|---|---|---|---|---|---|---|
| 11 Com b | 11 Com | 2 | 1 | Radial Velocity | 2007 | Xinglong Station | 0 | $323.21^{+0.06}_{-0.05}$ |
| 11 UMi b | 11 UMi | 1 | 1 | Radial Velocity | 2009 | Thueringer Lande | 0 | 516.21997±3.20000 |
| 14 And b | 14 And | 1 | 1 | Radial Velocity | 2008 | Okayama Astroph | 0 | $186.76^{+0.11}_{-0.12}$ |
| 14 Her b | 14 Her | 1 | 2 | Radial Velocity | 2002 | W. M. Keck Obser | 0 | $1766.41^{+0.67}_{-0.68}$ |

Figure 2. Full exoplanet database

| | A | B | C | D | E | F | G | H | I |
|---|---|---|---|---|---|---|---|---|---|
| 1 | planet_id | pl_name | hostname | pl_letter | hd_name | hip_name | tic_id | gaia_dr2_id | gaia_dr3_ic |
| 2 | 1 | 11 Com b | 11 Com | b | HD 107383 | HIP 60202 | TIC 72437047 | Gaia DR2 39469 | Gaia DR3 3 |
| 3 | 2 | 11 UMi b | 11 UMi | b | HD 136726 | HIP 74793 | TIC 230061010 | Gaia DR2 16967 | Gaia DR3 1 |
| 4 | 3 | 14 And b | 14 And | b | HD 221345 | HIP 116076 | TIC 333225860 | Gaia DR2 19201 | Gaia DR3 1 |
| 5 | 4 | 14 Her b | 14 Her | b | HD 145675 | HIP 79248 | TIC 219483057 | Gaia DR2 13852 | Gaia DR3 1 |
| 6 | 5 | 16 Cyg B b | 16 Cyg B | b | HD 186427 | HIP 96901 | TIC 27533327 | Gaia DR2 21355 | Gaia DR3 2 |
| 7 | 6 | 17 Sco b | 17 Sco | b | | HIP 79540 | TIC 135596590 | Gaia DR2 43424 | Gaia DR3 4 |
| 8 | 7 | 18 Del b | 18 Del | b | HD 199665 | HIP 103527 | TIC 354489950 | Gaia DR2 17567 | Gaia DR3 1 |

Figure 3. Downloaded the data and opened it into a Google Sheets file, and added the primary key, planet_id

There are 5 questions I want to research, looking at this data.

1. Other than Earth-like size, like a similar radius, what other parameters or constraints should I include that are also included in the column data table, to find an Earth-like planet? Why are those columns significant when looking for Earth-like planets?
2. How many planets from our data fulfill all these parameters to find an Earth-like planet? Does it match what was expected? Why or why not?
3. What would a visit to the planet be like according to the data?
4. Most, if not all, of the planets that have been discovered reside in our own galaxy, the Milky Way. According to the data we have, estimate how many Earth-like planets there are in the Universe based on the number of galaxies that have been discovered? What does this number say about us?
5. What are the biases and limitations of the data, and these questions? And what could it mean for our final estimation?

From there I started preprocessing and cleaning the Earth-like planet data in Jupyter Lab. The data is open, real, and not too simple. There are way too many columns, 320, including many that aren't helpful for this project. We need to find the critical columns that we need and dispose of the rest, but in order to do that, we need to answer research question 1 listed above. "Other than Earth-like size, like a similar radius, what other parameters or constraints should I include that are also included in the column data table, to find an Earth-like planet? Why are those columns significant when looking for Earth-like planets?"

| Name box (Ctrl + J) | | B | C |
|---|---|---|---|
| 1 | column_id | abbrev_name | full_name |
| 2 | 1 | planet_id | Planet ID |
| 3 | 2 | pl_name | Planet Name |
| 4 | 3 | hostname | Host Name |
| 5 | 4 | pl_letter | Planet Letter |
| 6 | 5 | hd_name | HD ID |
| 7 | 6 | hip_name | HIP ID |
| 8 | 7 | tic_id | TIC ID |
| 9 | 8 | gaia_dr2_id | Gaia DR2 ID |
| 10 | 9 | gaia_dr3_id | Gaia DR3 ID |
| 11 | 10 | sy_snum | Number of Stars |
| 12 | 11 | sy_pnum | Number of Planets |
| 13 | 12 | sy_mnum | Number of Moons |
| 14 | 13 | cb_flag | Circumbinary Flag |
| 15 | 14 | discoverymethod | Discovery Method |
| 16 | 15 | disc_year | Discovery Year |
| 17 | 16 | disc_refname | Discovery Reference |
| 18 | 17 | disc_pubdate | Discovery Publication Date |
| 19 | 18 | disc_locale | Discovery Locale |
| 20 | 19 | disc_facility | Discovery Facility |
| 21 | 20 | disc_telescope | Discovery Telescope |

Figure 4. The column data sheet includes full names for each column. I highlighted the ones most critical

We went through the data sheet and manually highlighted the ones that seemed the most critical for our project. Then, in a Jupyter notebook, we used the highlighted columns and filtered out the data to include only the critical columns in our column data set (20) and in our planets dataset (31).

```
[20]: #### Find critical columns in column data
critical_column_data=['planet_id','pl_name', 'hostname', 'pl_letter', 'sy_snum', 'sy_pnum',
                        'sy_mnum','discoverymethod', 'disc_year', 'disc_facility',
                        'pl_orbper','pl_orbsmax', 'pl_angsep' , 'pl_rade', 'pl_bmasse',
                       'pl_dens', 'pl_orbeccen', 'pl_insol', 'pl_eqt', 'pl_orbincl', 'pl_tranmid',
                       'pl_trandep', 'pl_trandur', 'pl_rvamp', 'st_spectype', 'st_rad', 'st_mass',
                       'st_met', 'st_lum', 'st_logg', 'st_age', 'st_dens', 'st_vsin', 'st_rotp',
                       'st_radv', 'ra', 'dec', 'sy_dist']

new_column_data = column_data[column_data['abbrev_name'].isin(critical_column_data)]

#new_column_data = column_data[critical_column_data]
print(new_column_data)

     column_id      abbrev_name                                full_name
0            1        planet_id                                Planet ID
1            2          pl_name                              Planet Name
2            3         hostname                                Host Name
3            4        pl_letter                            Planet Letter
9           10          sy_snum                          Number of Stars
```

Figure 5. Critical columns listed and used to redefine the column dataset

```
[31]: #### Find critical columns
critical_planet_columns=['planet_id','pl_name', 'hostname', 'pl_letter', 'sy_snum', 'sy_pnum',
                         'sy_mnum','discoverymethod', 'disc_year', 'disc_facility',
                         'pl_orbper','pl_orbsmax', 'pl_angsep' , 'pl_rade', 'pl_bmasse',
                        'pl_dens', 'pl_orbeccen', 'pl_insol', 'pl_eqt', 'pl_orbincl', 'pl_tranmid',
                        'pl_trandep', 'pl_trandur', 'pl_rvamp', 'st_spectype', 'st_rad', 'st_mass',
                        'st_met', 'st_lum', 'st_logg', 'st_age', 'st_dens', 'st_vsin', 'st_rotp',
                        'st_radv', 'ra', 'dec', 'sy_dist']

new_planet_data = planet_data[critical_planet_columns]
print(new_planet_data)
```

```
      planet_id     pl_name  hostname pl_letter  sy_snum  sy_pnum  sy_mnum  \
0             1    11 Com b    11 Com         b        2        1        0
1             2    11 UMi b    11 UMi         b        1        1        0
2             3    14 And b    14 And         b        1        1        0
3             4    14 Her b    14 Her         b        1        2        0
4             5  16 Cyg B b  16 Cyg B         b        3        1        0
...         ...         ...       ...       ...      ...      ...      ...
```

Figure 6. Used the critical columns to redefine the planets dataset

After cleaning the columns, we looked at the rows or entries. There are hundreds of rows with missing data. We need to clean the data so that only the necessary columns with entries for every row show. To do this, we found out the sum of missing data from each column shown on the left. Then we used those columns to drop all the rows or planets with missing data, so when we look for sums of missing data from each column, we are left with zero for each column.

```
planet_id             0
pl_name               0
hostname              0
pl_letter             0
sy_snum               0
sy_pnum               0
sy_mnum               0
discoverymethod       0
disc_year             1
disc_facility         0
pl_orbper           337
pl_orbsmax          315
pl_angsep           344
pl_rade              50
pl_bmasse            31
pl_dens             139
pl_orbeccen         938
pl_insol           1842
pl_eqt             1565
pl_orbincl         1522
pl_tranmid         1266
pl_trandep         1777
pl_trandur         1683
pl_rvamp           3667
st_spectype        3847
st_rad              317
st_mass               8
st_met              553
st_lum              311
st_logg             321
st_age             1314
st_dens             586
st_vsin            3881
st_rotp            5268
st_radv            3772
ra                    0
dec                   0
sy_dist              27
```

```
[45]: #calculating rows with missing values again after dropping planets missing data
#all are zeros
missing_column_data = new_planet_data.isna()
total_columns = missing_column_data.sum(axis=1)
#left with 162 entries, or 162 planets that have data in all the required columns
print(total_columns)
```

```
36      0
54      0
55      0
110     0
209     0
       ..
6078    0
6097    0
6100    0
6107    0
6141    0
Length: 162, dtype: int64
```

Figure 7. Checked for missing values in the main dataset, new_planet_data

```
[33]: #we need to drop all planets with missing values under a column, starting with pl_orbper
required_columns = ['pl_orbper','pl_orbsmax', 'pl_angsep' , 'pl_rade', 'pl_bmasse',
                    'pl_dens', 'pl_orbeccen', 'pl_insol', 'pl_eqt', 'pl_orbincl', 'pl_tranmid',
                    'pl_trandep', 'pl_trandur', 'pl_rvamp', 'st_spectype', 'st_rad', 'st_mass',
                    'st_met', 'st_lum', 'st_logg', 'st_age', 'st_dens', 'st_vsin', 'st_rotp',
                    'st_radv', 'ra', 'dec', 'sy_dist']

new_planet_data.dropna(subset= required_columns, inplace=True)

#calculating columns with missing values again after dropping planets missing data
#all are zeros
missing_row_data=new_planet_data.isna()
total_rows=missing_row_data.sum()
print(total_rows)
```

Figure 8. Counted columns with missing values

Figure 9. Dropped planets with missing values

When we looked at the number of rows or planets left, we found that we had 162 planets with complete data, as shown in Figure 7. This is a good number to start with for the Earth-like constraints we will use later on.

The other dataset, called planets_column_data, doesn't contain missing values because of its nature. Every column has a full name column. In other words, this dataset is a list of abbreviations and their full name and is used to understand the abbreviated columns in the main planet dataset. It's a pretty simple table with only 3 columns: ID, abbreviation, and full name, as shown below.

```
[22]: #redoing the planet's column id, 1 to length of column, 1 to 38
new_column_data['column_id']=range(1, len(new_column_data) + 1)
print(new_column_data)
```

```
   column_id      abbrev_name                          full_name
0          1        planet_id                          Planet ID
1          2          pl_name                        Planet Name
2          3         hostname                          Host Name
3          4        pl_letter                       Planet Letter
4          5          sy_snum                     Number of Stars
5          6          sy_pnum                  Number of Planets
6          7          sy_mnum                    Number of Moons
7          8  discoverymethod                   Discovery Method
8          9        disc_year                     Discovery Year
9         10    disc_facility                 Discovery Facility
```

Figure 10. Column data set shown above

# Stage 2. Model your data

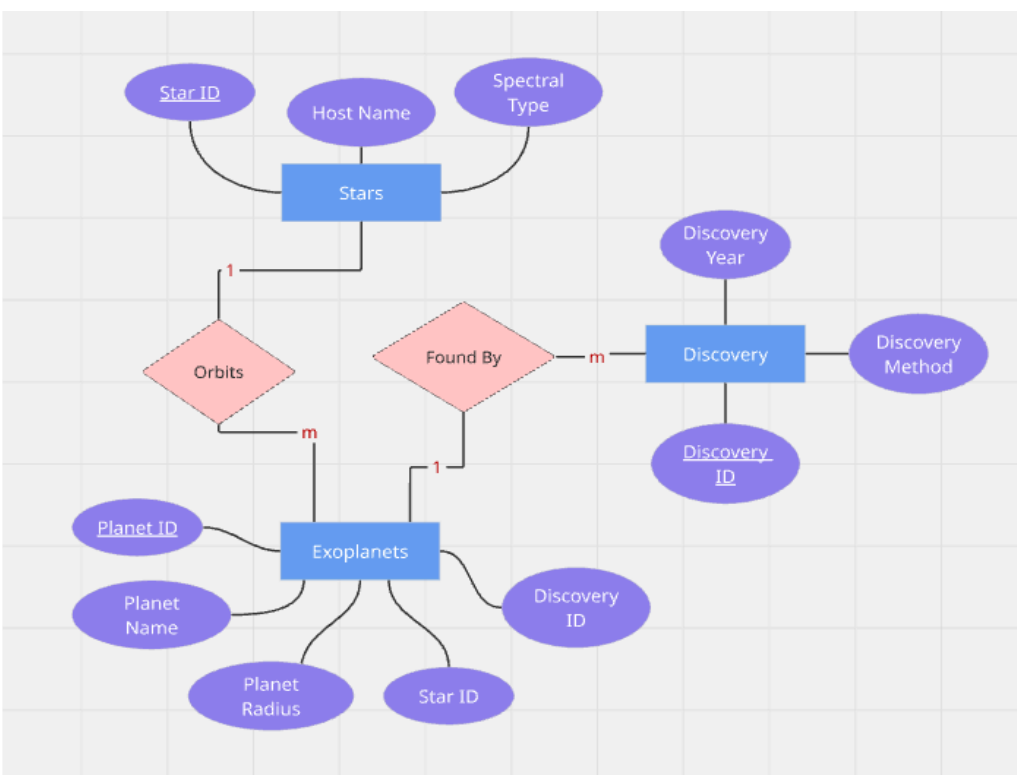



Figure 11. First Basic Entity Relationship Diagram

Entities: Stars, Exoplanets, Discovery
Primary Keys: Planet ID, Star ID, Discovery ID
Exoplanets Attributes: Planet ID, Planet Name, Planet Radius, etc.
Stars Attributes: Star ID, Host Name, Spectral Type, etc.
Discovery Attributes: Discovery ID, Discovery Year, Discovery Method, etc.
Relationships: Exoplanets and Stars- m:1, Exoplanets and Discovery- 1:m

When the model was started, the quick realization that having two datasets was unnecessary and made data handling more complicated than it needed to be was found. It was a lot easier and more direct to just rename the columns in Jupyter Lab and have the full names ready to go in the main dataset for exoplanets. This also made modeling the ER diagram easier. The first basic ER diagram created is shown above. There were a few basic errors caught in the first ER diagram, later on. Firstly, foreign keys discovery ID and star ID don't need to be added as bubbles under other entities. The lines show the relationships between entities and primary keys. Secondly, there can be many exoplanets per discovery, so a many to one relationship should be shown between exoplanets and discovery. Thirdly, Host Name usually refers to the solar system or system name, so it should be under a different entity. Fourthly, a fourth entity called Solar System should be included for system attributes that don't describe the star directly. Finally, all the attributes should be added to the final draft of the ER diagram.

I created a final draft of the ER Diagram shown below that fixes all the problems mentioned and showcases the four main entities in the data: **Exoplanets, Stars, Discoveries, and Solar System**. There are 38 columns of data plus the foreign keys underlined, and each entity has a bubble, making 41 bubbles total. This model could be used to understand the relationships between entities and how the database will be organized. There is a many-to-one relationship between exoplanets and stars, in which multiple exoplanets can orbit one star. There is a many-to-one relationship between exoplanets and discoveries, where multiple exoplanets can happen with one discovery moment, facility, method, etc. There is a many-to-one relationship between exoplanets and solar systems where there are multiple planets per solar system. Finally, there is a many-to-one relationship between stars and solar systems, where a system can have more than one star.

In the ER diagram shown below, we included all the attributes for each entity for a complete diagram and showed all the relationships between entities. Star ID, Discovery ID, and System ID will be used as foreign keys in Exoplanets when creating the database using SQL tables. The new diagram shows how the data will be normalized by splitting the flat data table into separate entities for a relational model database. The data will also be normalized, and this is shown in the ER diagram. Attributes don't have to be repeated more than once in multiple entities. Certain bubbles didn't directly depend on stars like distance. Distance is the distance between Earth and the system the planet is part of. If there is an exoplanet that belongs to a certain system, the distance from the Earth to that system won't be repeated twice for stars and the solar system. Systemic Radial Velocity is a bubble under Stars and not the solar system because it uses the star's light to find the velocity at which the system is moving away. Since it depends on the star, it's listed under Stars. When data is organized into separate entities, we avoid the redundancy seen in a flat dataset table. For example, we avoid listing the star data and the solar system data multiple times for multiple exoplanets found in the same system and star. Instead of repeating this data, it is normalized so the data about the star or the solar system only has to be listed once. This makes it easier to grow and manipulate the database as it grows.

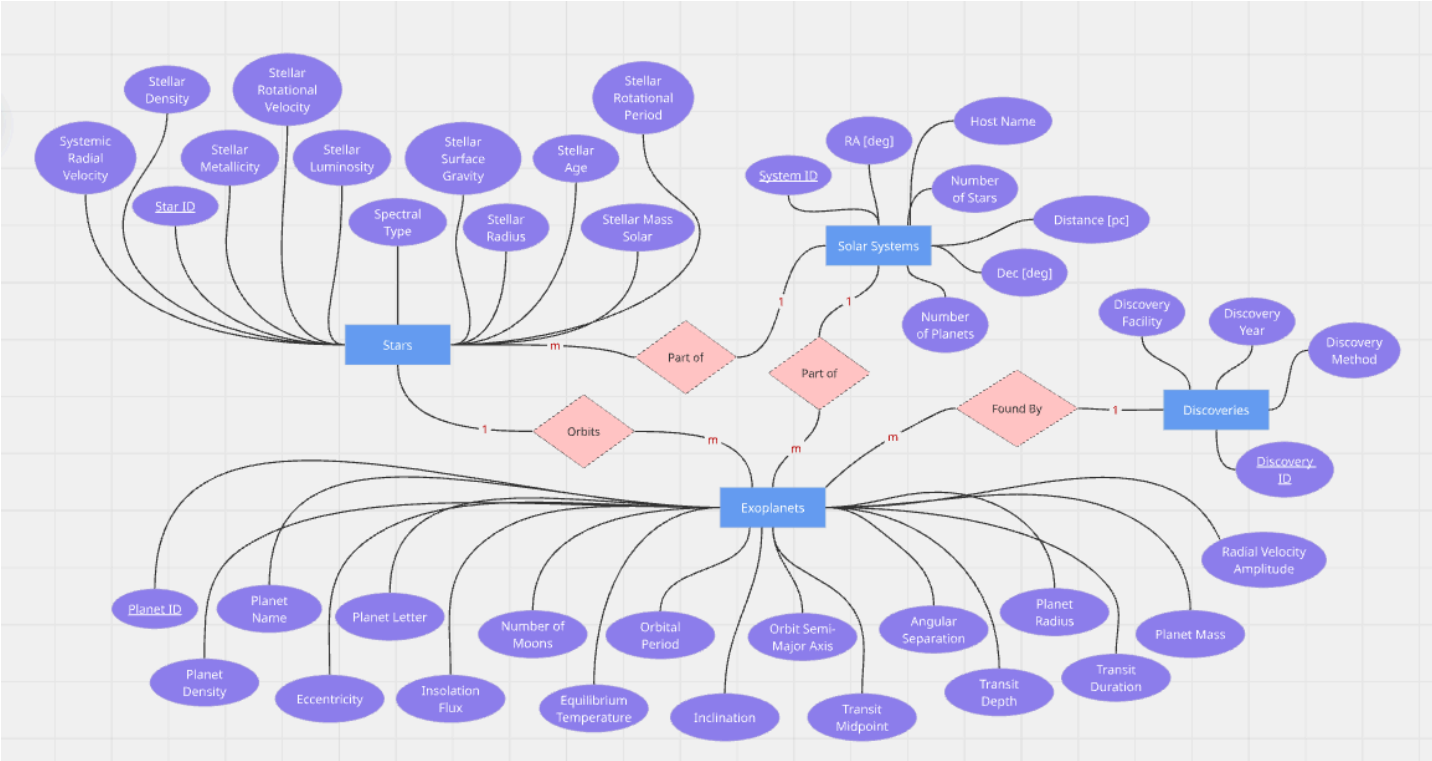



Figure 12. Final ER Diagram

The final part of stage two, as shown in figures 13 to 20, is to add the foreign keys in Jupyter Lab and to rename all the original column names to their actual Full Names to make the implementation on SQL better. We can do this by using the rename() Python function to rename each column and by adding the foreign keys as columns in the exoplanet dataset. We can create 3 new datasets for each entity and then add the appropriate data to the correct dataset, as shown below.

```
## instead of having two datasets, make the columns names into their full names for more straight forward data handling

#rename columns here
new_planet_data = new_planet_data.rename(columns={'planet_id': 'Planet ID', 'pl_name': 'Planet Name',
                                                  'hostname': 'Host Name', 'pl_letter': 'Planet Letter',
                                                  'sy_snum':'Number of Stars', 'sy_pnum':'Number of Planets',
                                                  'sy_mnum':'Number of Moons','discoverymethod':'Discovery Method',
                                                  'disc_year':'Discovery Year','disc_facility': 'Discovery Facility',
                                                  'pl_orbper':'Orbital Period [days]','pl_orbsmax':'Orbit Semi-Major Axis
```

Figure 13. Renamed columns to their full names as found in the columns data set

```
                                                  sy_dist : Distance [pc] })
print(new_planet_data)
```

```
     Planet ID Planet Name Host Name Planet Letter  Number of Stars  \
0            1    55 Cnc e    55 Cnc             e                2
1            2    AU Mic b    AU Mic             b                1
2            3    AU Mic c    AU Mic             c                1
3            4  CoRoT-18 b  CoRoT-18             b                1
```

Figure 14. Rename columns result

```
[19]: ##add new columns to dataset, Star ID, Discovery ID, Solar System ID

#variables for number of columns and rows or current data
number_columns = len(new_planet_data.columns)
number_rows = len(new_planet_data)

## add new columns using data.insert(location, column name, rows) at the end of the data
new_planet_data.insert(number_columns, 'Star ID', range(1,  number_rows + 1))

new_planet_data.insert(number_columns + 1, 'System ID', range(1, number_rows + 1))

new_planet_data.insert(number_columns + 2, 'Discovery ID', range(1, number_rows + 1))

print(new_planet_data)
```

Figure 15. Added foreign keys to the end of the main dataset, exoplanets

```
[162 rows x 41 columns]
```

```
[20]: ## Create a new dataset for Stars, Solar System, and Discoveries

stars_data = new_planet_data['Star ID']
print(stars_data)

system_data =  new_planet_data['System ID']
print(system_data)

discoveries_data =  new_planet_data['Discovery ID']
print(discoveries_data)
```

```
0        1
```

Figure 16. Created new data sets with the foreign key columns we just created in exoplanets

```
[21]: ## Add data to new data sets
#i need to find the columns, then i need add the columns to the new dataset

#Stars
#Spectral Type, Stellar Radius [Solar Radius],
#Stellar Mass [Solar mass], Stellar Metallicity [dex],
#Stellar Luminosity [log(Solar)], Stellar Surface Gravity [log10(cm/s**2)],
#Stellar Age [Gyr], Stellar Density [g/cm**3], Stellar Rotational Velocity [km/s]
#Stellar Rotational Period [days], Systemic Radial Velocity [km/s],

#find columns and assign to variable
stars_columns=['Star ID','Spectral Type', 'Stellar Radius [Solar Radius]',
'Stellar Mass [Solar mass]', 'Stellar Metallicity [dex]',
'Stellar Luminosity [log(Solar)]', 'Stellar Surface Gravity [log10(cm/s**2)]',
'Stellar Age [Gyr]', 'Stellar Density [g/cm**3]', 'Stellar Rotational Velocity [km/s]',
'Stellar Rotational Period [days]', 'Systemic Radial Velocity [km/s]']

#redfine stars_data using new varable with column names
stars_data = new_planet_data[stars_columns]
print(stars_data)
```

```
     Star ID Spectral Type  Stellar Radius [Solar Radius]  \
0          1          G8 V                         0.9430
1          2          M1 V                         0.8620
2          3          M1 V                         0.8620
```

Figure 17. Added more data to the stars, system, and discoveries dataset using the columns from exoplanets

```
[25]: #count of entries before drop_duplicates
star_entries_before = len(stars_data)
print( 'stars before: ', star_entries_before)

stars_data.drop_duplicates(subset=['Spectral Type', 'Stellar Radius [Solar Radius]',
'Stellar Mass [Solar mass]', 'Stellar Metallicity [dex]',
'Stellar Luminosity [log(Solar)]', 'Stellar Surface Gravity [log10(cm/s**2)]',
'Stellar Age [Gyr]', 'Stellar Density [g/cm**3]', 'Stellar Rotational Velocity [km/s]',
'Stellar Rotational Period [days]', 'Systemic Radial Velocity [km/s]'], inplace=True)

#after drops
star_entries_after = len(stars_data)
print( 'stars after:', star_entries_after)

#difference
star_entries_dropped = star_entries_before - star_entries_after
print("Duplicates Removed:", star_entries_dropped)

stars before:  162
stars after: 131
Duplicates Removed: 31
```

Figure 18. Duplicates were found and removed from each dataset or entity, starting with stars

```
missing_values = discoveries_data.isna()

row_total=missing_values.sum()
print(row_total)
print('_____________________Discoveries_____________________')

System ID                                   0
dtype: int64
_____________________Stars_____________________
System ID            0
Host Name            0
Number of Stars      0
Number of Planets    0
RA [deg]             0
Dec [deg]            0
Distance [pc]        0
dtype: int64
_____________________Solar System_____________________
Discovery ID         0
Discovery Method     0
Discovery Year       0
Discovery Facility   0
dtype: int64
_____________________Discoveries_____________________
```

Figure 19. No missing values were found in each dataset

```
[30]: #store cleaned datasets in new CSV files

      #save stars data
      cleaned_stars_data = stars_data.copy()
      cleaned_stars_data.to_csv('cleaned_stars_data.csv', index=False)
      print(cleaned_stars_data)
```

Figure 20. After preprocessing and cleaning, each dataset for each entity was saved into a CSV file for later use for creating the database using SQL

The main exoplanet data was split into four entities or four datasets. For each dataset, we defined the critical columns we wanted from the exoplanets dataset and then redefined the new dataset using the critical columns we listed, so all the columns listed would be taken from the original dataset and copied over to the new dataset. After taking the critical columns for stars, systems, and discoveries, we were left with exoplanets, so we could redefine exoplanets in the end without missing columns.

We used drop_duplicates() to remove duplicate entries for the same star, system, discovery, or planet. We used isna() to find missing values. We added foreign keys System ID, Star ID, and Discovery ID to the exoplanet dataset, and we added a foreign key, System ID, to the Stars dataset to link it to the solar system it's in.

After all the cleaning, processing, and splitting data into datasets, we normalized our data and achieved 3NF, 3rd Normal Form. Then all the datasets were saved into new CSV files for later use in the next stage of the database creation. All the tables will include the full names of each attribute and the primary keys and foreign keys.